
Known Issues and Lessons Learned

This document tracks the significant technical problems encountered while building this stack, their root causes, and the workarounds currently in place. It exists so that (a) the thesis discussion section has accurate detail to draw from, and (b) anyone picking this repository up later — a committee member, a future student, or future-me — doesn't have to rediscover the same things from scratch.

1. The `DiffAssertValues` `patricia trie` bug

Status: Worked around, not fixed upstream. This was the central technical blocker for most of the implementation period.

Symptom: Certain transactions — specifically `deploy_account` and `INVOKE` transactions — caused a `DiffAssertValues` error during SNOS execution when a contract's storage trie made its *first* transition from a single entry to two entries (the first binary split in the Patricia trie).

Root cause: Conflicting old-state and new-state edge length encodings in Cairo VM memory at `patricia.cairo:315`, traced to `format_commitment_facts` in the Karnot `generate-pie` crate (snos commit `9d3ad53`). `L1_HANDLER` transactions (bridge deposits) are unaffected — only `deploy_account` and `INVOKE` trigger it.

Workaround: Batch new-account onboarding via simultaneous bridge deposits. When two or more accounts are funded in the same L1 block (e.g. via explicit, consecutive nonces), their first L2 writes land in the same L2 block, which avoids triggering the single-entry-to-two-entry split in isolation. This was validated and used successfully — e.g. UAV2's `deploy_account` transaction (block 114) settled end-to-end without triggering the bug.

Thesis framing: Rather than a pure limitation, this can be presented as a constraint that happens to align with naval deployment doctrine — units are typically onboarded in formations/groups rather than strictly one at a time, so the batching requirement is not an unreasonable operational constraint.

Key contacts for follow-up: Jean-Baptiste Caron and Mohit Dhattarwal (Karnot / Madara Alliance).

2. Orchestrator upgrade attempt (v0.10.2 → v0.11.0-alpha.3)

Status: Reverted. Stack runs on `madara-orchestrator:v0.10.2`.

An upgrade to `v0.11.0-alpha.3` was attempted (built locally from the `v0.11.0-alpha.3` tag of the Madara monorepo). The new orchestrator version introduced `AggregatorBatching` errors — “Failed to parse REST response: error decoding response body” — with no requests reaching the `stwo-gateway` at all. This indicates an API contract change between orchestrator versions that the gateway's SHARP-compatible implementation does not yet account for.

Decision: Reverted to `v0.10.2` to preserve a working state ahead of the thesis deadline. Investigating and updating `stwo-gateway` to match the new contract was deferred as future work rather than risking stability this close to submission.

3. Full-node write forwarding requires explicit configuration

Status: Resolved (configuration gap, not a Madara limitation).

Initial assumption was that Madara full nodes (`madara-fullnode`, `madara-vessel-frigate`) cannot forward write transactions to the sequencer at all. This is incorrect — Madara documents this as a built-in feature (“write methods are forwarded to the Sequencer and are not executed by Madara”). The actual issue was that two CLI flags, required on both sides, were missing from `compose.yaml`:

- Sequencer side: `--gateway-trusted-add-transaction-endpoint`
- Full-node side: `--validate-then-forward-txs-to <sequencer-gateway-url>`

Without these, a full node attempting to forward a write transaction fails with `Error calling HTTP client: client error (Connect)`, because it has no configured target to forward to.

Lesson: Always check `madara --help` against the specific version in use before concluding a feature doesn't exist — flag names and defaults have changed across releases.

4. The forwarding mechanism is not the target architecture

Status: Documented as a deliberate simplification; see `docs/ARCHITECTURE.md`.

`--validate-then-forward-txs-to` requires a fixed, pre-configured target address. It is a pragmatic mechanism for demonstrating that a non-sequencer node can accept transaction submissions, but it is **not** the dynamic, permissioned, rotating-sequencer L2 network described in the thesis' target architecture. True dynamic membership — nodes discovering peers and propagating transactions with no pre-configured channel, plus a rotating/elected sequencer — requires the peer-to-peer gossip layer that Madara's own roadmap describes as in progress but not yet shipped in v0.10.2.

5. Fresh restarts wipe all state; `sncast account list` is misleading

Status: Known behavior, not a bug — documented here to save debugging time.

Running `./up.sh` without `--resume` wipes `data/` and `output/` and re-bootstraps from genesis. The `sncast account list` command, however, reads from `~/.starknet_accounts/starknet_open_zeppelin_accounts.json` on the local workstation, which is never touched by `up.sh`. After a fresh restart, this file will still report previously-deployed accounts as `deployed: true`, even though they no longer exist on the new chain. This has caused confusion more than once — always check `output/base_addresses.json` and `output/madara_addresses.json` after any restart to confirm whether you got a fresh chain or a resumed one, and re-deploy accounts/contracts accordingly.

Useful invariant: the STRK fee token address, core contract address, and StarkGate bridge address are all **deterministic** — they do not change across fresh bootstraps, since they derive from the bootstrapper configuration rather than from chain history. Only account and contract-instance addresses change on a fresh start.

6. Grafana “No data” after a stack restart

Status: Resolved.

A dashboard provisioned from a JSON file containing the Grafana template variable `${DS_PROMETHEUS}` will silently fail to resolve that variable against the actual provisioned datasource UID, producing “No data” on every panel even though the full Madara → OTEL collector → Prometheus pipeline is healthy. Confirmed via `docker logs`, the Prometheus `/api/v1/targets` endpoint, and direct `/api/v1/query` calls that the underlying metrics pipeline was working throughout — the break was purely in the dashboard JSON's datasource reference.

Fix: Replace `${DS_PROMETHEUS}` with the literal datasource UID (visible via `GET /api/datasources` on the Grafana API) directly in the dashboard JSON, then restart the Grafana container to reload the provisioned dashboard.